TraceWeave-Core

TraceWeave-Core Project

August 12, 2026

CONTENTS

What this project is

- The background problem
- Where this project sits in a larger program

The hypothesis

- H2a, stated plainly
- How it gets measured
- The number is deliberately not pre-committed
- The two sibling hypotheses

The honesty machinery

- Pre-registration
- Rules that deliberately shrink the headline number
- Comprehension normalization, a subtle one
- Pinning: making “the same repo” mean something
- The placebo arm

How things are classified

- The density scorer
- The edge-type taxonomy
- Secondary measurements that come almost free

The plan

- Six phases in twelve weeks
- The funnel

- The single biggest schedule decision
- Five amendments to the original procedures
- Deliberately out of scope
- What gets released

Status

- What exists and works
- The discovery run
- The one question that run was designed to answer
- Timings
- Things that went wrong, recorded rather than hidden
- Seven findings from Phase 0

In one paragraph

What this project is

The background problem

Companies hold huge amounts of old software they would like to modernize: move it to a new language, a new framework, or a newer version of the one they have. There is a lot of excitement about using AI models to do that work automatically.

For an AI to safely change one piece of code, it needs to know **what else that piece touches**. If you rename a function, what breaks? To answer that, tools build a **call graph**: a map where each box is a function and each arrow means “this function calls that one.” The AI is handed the relevant slice of that map along with the code itself.

The map is built by a **static analyzer**, a program that reads source code without running it. The analogy is tracing a road map by reading street signs rather than by driving the roads. That works well when the code says plainly `send_email()`. It works badly when the code says, in effect, *”look up whatever the config file names, and call that.”* The name only exists while the program is running. On paper, the arrow simply is not there.

Python is full of exactly this kind of code: plugin systems, dependency injection, dynamic imports, reflection.

A systematic literature review (the `final_paper.pdf` that sits at the root of this project) surveyed the field and concluded that **the bottleneck for AI-driven modernization is not the AI, it is the map**. The graph does not contain the information the model needed. Section VIII.B of that review puts it directly: “the bottleneck is therefore graph construction and the amount of information that can be represented.”

That is a claim the review *argued*. Nobody has **measured** it.

Where this project sits in a larger program

The parent document, `research_plans.md`, proposes two separate 40-week research programs:

- **Direction 1 (GRAFT)**: fine-tune a small model with deterministic scaffolding and beat a frontier model on repository migration at under 10% of the cost.
- **Direction 2 (TraceWeave)**: fuse runtime traces into the code property graph and show that the missing structure was the real problem.

This project, **TraceWeave-Core**, is a 12-week slice of Direction 2. It takes roughly 30% of that program, chosen so the 30% is a self-contained paper rather than one third of every milestone.

Direction 2 was chosen to run first, against the parent document's own sequencing advice, for three reasons:

1. **Direction 2 has a guaranteed floor; Direction 1 does not.** The core question here is a measurement. Direction 1's headline is a *comparative* claim whose truth depends on a frontier-model baseline and an API price curve that will both move before submission.
2. **This work de-risks Direction 1 cheaply.** If static graphs turn out to be nearly complete, Direction 2 dies for roughly zero dollars in model spend, and Direction 1 learns that its graph slices are a sound foundation. If they miss 30%, Direction 1 learns its slices are silently lossy. Nothing in Direction 1's early milestones returns comparable information.
3. **Cost asymmetry.** This project needs no fine-tuning and no distillation corpus. Direction 1 gates on a frontier-model run across training repositories plus QLoRA training, the single largest irreversible spend, committed *before* knowing whether the representation being fed to the model is adequate.

The hypothesis

H2a, stated plainly

> Static call-graph tools miss a substantial fraction of the calls that actually happen when a real Python program runs, and the misses are **not spread evenly**. They cluster in a few identifiable kinds of code: reflective dispatch, dynamic imports, and plugin or dependency-injection wiring.

There are two halves, and the second matters more than the first.

- **The quantity.** How big is the hole? 5%? 30%?
- **The shape.** *Where* is the hole? If the misses concentrate in three named constructs, you can tell tool-builders exactly what to fix. If they are diffuse, the finding is much weaker.

How it gets measured

The method is a comparison between two maps of the same program.

Map	How it is made	What it represents
Static (S)	Read the source code without running it	What today's tools can see
Runtime (R)	Actually run the program with instrumentation recording every call	What actually happened

Everything in R that is not in S is a **miss**.

```
recall(r)    = |S ∩ R| / |R|
miss_rate(r) = 1 − recall(r)
```

That number is reported overall, per stratum, per edge type, and per static tool. A mean is never shown on its own.

The number is deliberately not pre-committed

The parent research plan phrased this hypothesis as “misses at least 25%.” This project explicitly refuses that framing.

The frozen metric specification states the reasoning directly: a 14% miss rate concentrated entirely in dependency-injection and plugin-registration sites is a *better and stronger* result than a diffuse 30%, because it names a mechanism. The conditional claim is the publishable one.

And if the answer comes back small, the paper still exists. “Static extraction is more complete than the field assumes, except at these specific constructs” is a useful, publishable negative result.

This is the deeper strategic point. The project is a measurement **of programs, not of AI models**. Whatever number comes back is a fact. It requires no frontier-model baseline whose pricing and capability will move underneath it, and no fine-tuning spend. There is no outcome that wastes the work.

The two sibling hypotheses

The full Direction 2 program has three sub-hypotheses. This project measures the first in full and takes a bounded slice of the other two.

	Claim	Status here
H2a	Static extraction misses runtime edges, concentrated in identifiable constructs	**The primary target**
H2b	An AI given the fused graph resolves tasks whose dependency cone crosses a dynamic edge at 1.5x the rate of static-only, while performing *identically* where there are no dynamic edges	H2b-lite, Phase 4, bounded
H2c	Explicitly marking *unobserved* regions reduces confidently-wrong migrations	H2c-lite, first to be cut

H2b’s second half is the important half. The built-in placebo control is what isolates a representation gain from a generator gain.

The honesty machinery

The catch with a measurement like this is that almost every definitional choice can be nudged to make the headline number bigger. So the project's structure is largely an elaborate set of self-imposed handcuffs.

Pre-registration

The document defining *what counts as a miss* (`docs/metric_spec_h2a.md`) was frozen on 2026-07-27, **before a single line of measurement code was written**. Its stated reason: "Freezing them in advance is what makes the number a measurement rather than a search."

Changing it requires a version bump, a dated changelog entry, and re-running everything from scratch. Never a silent edit.

Three documents are frozen under that discipline:

Frozen	Version
`docs/metric_spec_h2a.md`	1.0
`docs/density_scorer_spec.md` and the thresholds in the scorer	1.0
`config/base_images.json` and `MANIFEST.md`	1.1

Rules that deliberately shrink the headline number

Calls into built-in and third-party code are excluded from the primary denominator. Static tools never claimed to track those, so counting them would manufacture an enormous fake miss rate. The specification calls this "the single easiest way to accidentally fabricate a headline number, so it is excluded by construction."

Decorators get "wrapper transparency." A Python decorator wraps a function. Naively comparing static and runtime produces two fabricated misses and one fabricated false alarm for every decorated call. The rule that collapses `caller → wrapper → f` back to `caller → f` was written before any extraction code existed.

The word “precision” is refused. If a static arrow was never seen at runtime, that could mean the tool was wrong, or it could mean no test happened to exercise that path. Traces cannot separate those two. So the quantity is labelled *workload-confirmed fraction* everywhere it appears. Genuine precision comes only from a manual audit, reported with its sample size and confidence interval, and is never extrapolated to a headline.

Runtime evidence is always described as a lower bound, conditional on which workloads were run. That wording is mandatory in every metric definition, every table caption, and every figure, not buried in a threats-to-validity section.

Recall is always conditioned on execution coverage. `fn_coverage` is the fraction of the repository’s functions that ran at least once. A high miss rate on a repository with 8% coverage means something very different from the same rate at 70%, so every recall figure is plotted against it.

No silent drops. Every repository that fails at any stage stays in the record with a reason. A repository that vanishes quietly creates survivorship bias in a published benchmark.

No AI-as-judge anywhere in validation or in headline metrics. It is permitted only in failure *analysis*, clearly marked as such.

Comprehension normalization, a subtle one

Python 3.12 changed how list comprehensions are compiled (PEP 709 inlined them; generator expressions stayed separate). Without a normalization rule, the same code would produce a different set of nodes on a 3.12 repository than on a 3.10 one, and cross-version comparison would be invalid. The spec normalizes comprehensions into their enclosing function. This was written down before it could become a problem.

Pinning: making “the same repo” mean something

Because “the same repository” must mean the same thing months later, each one is identified by a mandatory five-part fingerprint:

```
(repo_url, sha, pyver, base_image_digest, lockfile_sha256)
```

All five are required. A repository missing any one of them is not in the corpus.

- **`sha`** is the exact commit. An earlier version of the process allowed this to be blank; the current version does not. The runner resolves and records it at clone time.
- **`base_image_digest`** is the container image's cryptographic digest, never a tag. Tags such as `python:3.12-slim` move; digests do not.
- **`lockfile_sha256`** is a hash of the exact dependency versions installed inside the container. This is described as "the single most important pin in the project." Without it, ordinary package-update drift on PyPI would silently invalidate every trace and every reproducibility claim.

Node identity is separately frozen as `(repo_url, sha, relative_path, qualified_symbol)`. **Line numbers are deliberately not part of it.** They move under reformatting and they differ between the static parse and the runtime frame. Lines are a disambiguation hint only.

The placebo arm

Repositories are sorted into LOW, MED, and HIGH bins by how much dynamic code they contain. LOW is where the theory predicts **no** effect at all.

Confirming that null is what converts "we added data and things got better" into a real causal claim about representation. The LOW bin has a hard floor of 10 repositories, and the plan states plainly that it is "the last thing cut, not the first."

How things are classified

The density scorer

Before anything is built or run, each repository is scored for how much dynamic code it contains: weighted dynamic-feature call sites per thousand lines of source, plus 1.5 per framework detected.

Stratum	Score
LOW	below 4.0
MED	4.0 to 10.0
HIGH	10.0 and above
TOO_SMALL	under 500 lines of source, ineligible regardless

The thresholds of 4.0 and 10.0 were calibrated once, on a 14-repository reference set, during the first week, before any corpus admission or measurement. They are frozen.

Three design decisions inside the scorer are worth knowing:

- **`getattr(o, "literal")` is weighted 0.3; `getattr(o, name)` is weighted 2.0.** The literal form is statically resolvable and is *not* a blind spot. Conflating the two is the commonest way to overstate dynamic density.
- **Source and tests are scored separately, and the band comes from source only.** Test suites use `getattr`, mocking, and monkeypatching for reasons unrelated to a project's architecture. Scoring them together made every project that uses pytest look HIGH.
- **The per-thousand-lines denominator is floored at 1000 lines.** Below that, dividing by size *amplifies* rather than normalizes. A 29-line test fixture scored 701 before the floor was added.

The bins exist for sampling balance and readable tables. **The continuous score is the actual analysis covariate**; miss rate is regressed on it. A repository just below a boundary is not meaningfully different from one just above it.

The edge-type taxonomy

Every call is classified by the *syntactic form of the call site*, read from the caller's source, and **not** by whether the static tool happened to find it. That ordering is what keeps "static misses X% of edges of type T" well-defined and non-circular.

Edge type	Call-site form	Expectation
`direct`	`f()` resolving to a module or local definition	Static should get nearly all. Sanity floor.
`method_static`	`obj.m()` where `obj`'s type is statically inferable	Static should mostly get these.
`method_dynamic`	`obj.m()` where the type is not inferable	Expected miss zone.
`super_dispatch`	`super().m()`	Expected over-approximation zone.
`reflective`	`getattr(o, name)()`, attribute interception	**Core target.**
`dynamic_import`	reached via `importlib.import_module`, `__import__`	**Core target.**
`eval_exec`	callee inside `eval` or `exec` output	Always becomes a phantom node.
`callback_dispatch`	a value pulled from a registry then called: `handlers[k]()`	**Core target.** The plugin and DI mechanism.
`decorator_apply`	import-time application of a decorator	Genuine edge; static tools do miss import-time effects.
`decorator_wrapper`	a call that lands in a decorator's wrapper	**Must not be counted as a miss.**
`subprocess_ipc`	process or socket boundary	Out of universe; recorded as an environment edge.

Secondary measurements that come almost free

- **Phantom rate.** How many runtime callees have no static counterpart *at all*: code created by `eval`, dynamically synthesized classes. This is a separate and stronger claim than the miss rate.
- **PyCG parse-failure rate.** How often the field's published standard static tool cannot even read contemporary Python. See the findings section.
- **Unresolved-dispatch census.** Statically unresolvable call sites that *no* workload ever exercised. These are the honest unknowns, and they seed H2c-lite.
- **Environment-contract gap.** Environment variables read at runtime versus those actually documented.

The plan

Six phases in twelve weeks

The project started 2026-07-27 with an internal deadline around 2026-10-26. Staffing is one engineer plus two to three human annotators.

Phase	Weeks	What happens
0	1	Freeze the rules, build the measuring instruments, stratify the queue
1	2–4	Get hundreds of repositories to actually install and run; build two independent static extractors
2	3–6	Build the tracer that watches programs run and records every call; normalize to Parquet
3	5–8	Map runtime events back to source, fuse the two graphs, produce **the H2a numbers**, go or no-go
4	8–11	H2b-lite and H2c-lite: one bounded, pre-registered experiment
5	10–12	Release the benchmark, write the paper

The Week 8 seam is deliberate. Phases 0 through 3, plus the feasibility survey, are already a complete paper. Phase 4 is additive by construction, so if time runs out it detaches cleanly instead of leaving everything half-finished.

The cut order was decided in advance, before anyone could feel schedule pressure: H2c-lite goes first, then Phase 4 entirely, then the LOW stratum down to its floor of 10. Never cut the placebo arm while keeping Phase 4. Never cut pinning.

The funnel

The hard part is that most repositories on GitHub will not install, let alone run. So the corpus narrows aggressively:

```
~450–600 candidates discovered and density-scored     (floor 300)
    ↓  automated triage: does it install? do tests run?
~120 that build cleanly with a pinned lockfile        (floor 60)
    ↓  humans record at least 3 replayable workloads each
~60 final corpus repositories                         (floor 40)
       LOW ~15 (hard floor 10) · MED ~20 (12) · HIGH ~25 (15)
```

The single biggest schedule decision

The build-triage step **was already effectively a script**: run a command, read the result line, look it up in a fixed table, apply a fixed recipe. There was no judgment in it, no reason for a 12-minute timebox, and no room for two annotators to disagree.

The workload-capture step is the opposite. It means reading a prose README, deciding whether a grey box on a web page is a runnable command, and finding the demo script nobody linked. It is irreplaceably human, and it is the true bottleneck.

So triage was automated and **every annotator was moved to workload capture**. At the standard-operating-procedure rate of 15 repositories per week per person, three people reach a 60-repository corpus in about two weeks of steady state instead of about four weeks serialized behind triage.

The 10% double-coding quality check was repurposed rather than dropped. Instead of annotator versus annotator, one annotator manually triages a 10% sample and it is compared against the automated classifier. That converts a quality metric into a *reportable validation of automated triage*, which the paper needs anyway. Annotators are explicitly told **not** to guess what the script said. A disagreement they found is worth more than an agreement they produced.

Five amendments to the original procedures

#	Amendment	Why it was urgent
A1	**Pin the environment.** Emit a dependency lockfile, the image digest rather than tag, and the resolved Python version for every non-failing repository.	The `python:3.12-slim` tag moves and PyPI resolves differently week to week. Every trace, and the entire reproducibility premise, silently rots without this. Cheap now; a redo of 300 annotated repositories later.
A2	**Stop discarding AMBER.** Repositories that install and run but whose tests fail are now valid input.	The hypothesis needs code that *runs*, not code that *passes*. A failing test still executes real code, which is exactly what the tracer watches. Excluding AMBER would have discarded an estimated 20 to 40% of a usable corpus.
A3	**Stratify before triage.** The density scorer gates entry to the queue, and rows are interleaved by bin.	Triage works strictly top to bottom. Unstratified input means the bin distribution is whatever chance gives, and the HIGH stratum comes out thin, which is the exact risk the parent plan names.
A4	**Delete the port-search step, automate the metadata step.**	The port search looked for Python ports of Python repositories, which is meaningless. It was inherited from Direction 1's Java-to-Python contamination screen. Together with the automation, roughly 30 person-days were returned.
A5	**Record exact seconds, not rounded to 5.**	Duration is the denominator for tracer-overhead ratios. Rounding a 3-second command to 5 makes that measurement meaningless.

Deliberately out of scope

Recorded so it does not get relitigated mid-project. All become future work in the paper.

- The GNN / graph-conditioned adapter arm
- The Java, Spring, and JVMTI secondary track

- The eBPF and strace system-telemetry layer
- End-to-end whole-repository migration, dependency-frontier ledgers, interface-freeze contracts, transactional rollback
- All of Direction 1: no fine-tuning, no distillation corpus, no frontier-model spend

What gets released

DynBench-Core: pinned repositories, replayable workloads, reference edge sets, and the harness. The field has benchmarks for migration *outcomes* but none for *graph completeness*, which the review names as the bottleneck. Benchmarks outlive methods, so this may end up the most-cited output.

A tracing-feasibility survey: a systematic record of how often, and why, real Python repositories cannot be built and traced at all. It is derived entirely from the failures, which is why failures are treated as data rather than as mess to be cleaned up.

Status

As of 2026-08-12: Phase 0 is complete. Phase 1 has not started.

That is roughly two and a half weeks into twelve. Phase 1 was scheduled for weeks 2 through 4, so that lane is about a week behind, though a Phase 1 prerequisite (candidate discovery) ran to completion on 2026-08-08 and consumed the time.

What exists and works

The measuring instruments are built, calibrated, frozen, and verified end to end. A suite of seven checks passes cleanly, including a full recalibration sweep that reproduces the frozen 6 HIGH / 4 MED / 3 LOW / 1 TOO_SMALL distribution exactly.

A real repository was taken through the whole pipeline as a proof: `click` triaged green in 93 seconds, produced a pinned nine-package lockfile with a sha256, and that exact environment was then reproduced from the lockfile with all 1,940 tests passing. Its commit hash was auto-recorded even though the input sheet supplied none.

The discovery run

The candidate pipeline ran on 2026-08-08 and delivered:

```
98,416 repos matched → 1,761 examined in detail → 600 eligible → 558
queued

strata:  HIGH = 135     MED = 177     LOW = 246
```

That is **558 queued against a target of 450 to 600**. Exclusions were 937 with no build file and 224 with no test suite. Of the 600 scored, 599 succeeded, 1 failed, and 41 came back TOO_SMALL. Density ran from 0.0 to 88.975 with a median of 4.433, and the thresholds were not touched.

Integrity: `sha_drift` was zero across all 600. Every repository was materialized at exactly its frozen commit, so scoring and triage will read identical trees. There was no silent fallback to a branch tip anywhere.

The queue file contains 1,761 rows rather than 558. Excluded repositories are retained with a reason and an empty queue position, per the no-silent-drops rule. Filtering on queue position gives the 558.

The one question that run was designed to answer

Would the HIGH bin come out too thin? HIGH is the bin the whole paper depends on, and a random sample of GitHub was expected to be far more LOW-heavy than a hand-picked calibration set.

```
[queue] survival needed to hit corpus floors:
         LOW   246 queued -> floor 10   needs   4.1% to survive
         MED   177 queued -> floor 12   needs   6.8% to survive
         HIGH  135 queued -> floor 15   needs  11.1% to survive
```

HIGH cleared the floor, but it is the binding stratum and it has no reserve. It needs 2.7 times the survival rate of LOW.

- **11.1%** clears the bare floor of 15.
- **16.7%** clears the 1.5x over-recruitment target the parent plan actually asks for, which is 22.5 surviving HIGH repositories. That is the number to plan against, not 11.1%.

Two structural facts about how HIGH sits in the queue:

- **HIGH is heavily front-loaded.** 58 of the 135 fall in the first 139 positions, which is 42% of that prefix. Triaging only the first quarter of the queue would require 25.9% HIGH survival to reach the floor; working the full queue drops that to 11.1%.
- **HIGH saturates at the 75% mark.** There are 135 at both the 75% and 100% points, meaning the final quarter of the queue contains no HIGH repositories at all. That is the pool being genuinely exhausted, not a bug in the interleaver. There is no way to extract more HIGH by triaging further down. Only a wider search would produce more.

If HIGH survival comes in under roughly 11%, the correct response is a fresh, wider discovery run, not more triage.

Timings

```
pool     95m39s   exit=1   <- crashed, see below
pool    279m05s   exit=0   <- full re-run from scratch
score     5m42s   exit=0
peak working-directory usage: 623 MB
```

Scoring ran at 12 parallel jobs: 600 repositories in 342 seconds, about 6.8 seconds of worker time each, which is right for a shallow single-commit fetch plus one parse. Disk was never a constraint at 623 MB against 333 GB free. The real ceiling was the Docker VM's 8.32 GB of RAM, which is what capped jobs at 12 rather than the 16 available cores.

Things that went wrong, recorded rather than hidden

The discovery run crashed after 95 minutes and lost all of that work. An unhandled network fault (`http.client.IncompleteRead`) matched none of the three exception handlers because it is an `HTTPException` rather than an `OSError`. The handoff document's claim that all three stages resume turned out to be **false for the search phase**: it runs entirely in memory and writes nothing until enrichment begins, so 95 minutes was unrecoverable and the re-run started from zero. Checkpointing just the seen-set and pending queue would have turned that into a few minutes.

The fix chosen was deliberately not the obvious one. Simply adding the exception to the existing handler list would have been *worse than the crash*: the caller treats a null return as "search failed" and breaks out of its page loop, so one truncated response would have silently dropped repositories from a star shard and biased the pool with no diagnostic at all. A retry with exponential backoff was used instead, falling back to the original contract only after five attempts. Server errors such as 403 and 404 are still deliberately not retried, because callers depend on seeing them immediately. The patch was tested against a mocked network layer for three cases before the re-run, and the original file was preserved for comparison. **In the successful 4h39m re-run the patch never fired**: zero retries, zero give-ups. The original crash was a genuine one-off transport fault, not something structural. The patch is insurance that was never billed.

The search stage fetches about ten times more data than it uses. It only needs a total count to decide whether to split a shard, and that arrives on page one, but it paginates all ten pages regardless. In this run 130 of the first 196 shards were splits, which is roughly 1,300 discarded pages and about two of the first three hours. Diagnosed, not yet fixed.

2,378 repositories were silently truncated from the low-star end. Twelve single-star-wide shards exceeded the search cap and hit a truncation branch. The star axis cannot be bisected below one-star width, so this is a hard floor of the sharding design rather than a tuning problem. Impact on this run is mild, because sampling is a seeded shuffle across all 98,416 matched repositories rather than star-ranked, so the effect is a slight under-weighting of the 50 to 62 star band. It would matter a great deal to anyone re-running with a lower star minimum. A second sharding axis, such as push date within a star bucket, would fix it properly.

One repository crashed the scorer. `Baekalfen/PyBoy` hit CPython’s recursion limit on a deeply nested syntax tree. Not an out-of-memory error, so it will reproduce on any hardware. The row was retained with a reason rather than dropped.

Two rate-limit hits during search, both recovered automatically by the existing sleep-and-retry path with no data lost. Noted only because the handoff asked for anything that looked wrong.

Docker was not running at first contact. Docker Desktop for Linux 4.78.0 installs as a *user* unit shipped disabled and inactive, so the context pointed at a socket that did not exist. The failure presents as a missing daemon rather than as a disabled service, which is worth a line in the setup instructions for Linux hosts.

Seven findings from Phase 0

These came from *running* Phase 0 rather than planning it.

1. **Docker Desktop cannot bind-mount `/tmp`.** All corpus paths moved under the home directory. A methodological consequence follows: Desktop runs containers inside a VM, so bind-mount input and output is materially slower than native. Absolute wall-clock timings from this host are inflated and must not be published as representative. **Ratios** (traced divided by untraced, measured under identical mount conditions) cancel the penalty and are the correct quantity. This is written down so nobody later reports raw seconds as an overhead figure.

2. **The density scorer's first run was useless.** Ten of 14 reference repositories landed in HIGH, giving no discrimination at all. Four false-positive classes were found and fixed before freezing:

   - `re.compile` was counted as `eval`-like at weight 3.0, which drove `requests` to a spurious HIGH
   - `@t.overload` and `@v.setter` were counted as registry decorators, so `click` scored 16.4 per KLOC on pure typing machinery
   - `importlib.metadata` was mislabelled as a framework, giving nearly every repository a spurious +1.5
   - `sys.modules` was counted on bare reads rather than only on mutation

3. **The scorer corrected two a-priori labels, and it was right.** `requests` was expected to be LOW; it scores mid-range because it genuinely does `locals()[package] = __import__(package)` and `importlib.import_module(lib)` in its compatibility layer, which is precisely the edge class this project is about. `toolz` similarly uses `globals()` in `curry`. Reported rather than quietly reconciled. It is mild evidence that the metric tracks something real.

4. **PyCG's latest release is 0.0.8 and it is years old.** It is the field's published standard static baseline, and it parses using the host's own parser, so modern syntax may crash it outright. Rather than hide this, **its parse-failure rate becomes a reported metric.** How often the field's standard tool cannot read contemporary Python is a completeness result about the state of the art in its own right. Every failure is logged with its traceback into the feasibility survey.

5. **Metadata extraction is about 90% automatable, not 100%.** GitHub returns an inconclusive licence for roughly one repository in ten. `celery` comes back as OTHER when it is really BSD-3-Clause, because its licence file has an unusual preamble. Only those rows go to a human, at about 30 seconds each.

6. **The original triage classifier has an ordering bug.** It tests one pattern first and unconditionally, so any repository whose metadata merely *mentions* it gets mislabelled regardless of the real failure. The buggy version is kept verbatim so the human-versus-script audit compares like with like, a corrected version runs alongside it, and **their disagreement rate is now a reportable number rather than a bug to hide.**
7. `records`**, one of the original practice repositories, is 495 source lines**, four under the 500-line gate. It is corpus-ineligible despite triaging green. Fine for practice, and a useful reminder that building successfully is not the same as being eligible.

In one paragraph

The field believes AI-driven code modernization is limited by incomplete code maps rather than by the AI itself, but nobody has measured the gap. This project measures it: building the map of a Python program two ways, by reading it and by running it, and quantifying what reading alone misses and where the misses concentrate. Every definition was frozen before any measurement code was written, so the number cannot be tuned after the fact. The instruments are built and validated, 558 candidate repositories are frozen at exact commits and queued, and the next step is putting them through automated build triage. The measurement itself lands around Week 8, in early October.